

Chapter 11

Iterators and the Standard Template Library

How to read these notes

- ★ marks material that is core for this class and worth knowing well.
- Content inside a **gray X box** is **not required for this class**. It is included for completeness and future reference.
- Functions listed under “reference only” do not need to be memorized, but knowing they exist helps a lot when writing code.

11.1 The Standard Template Library (STL)

- The **standard template library (STL)** is a well-documented C++ library of high-quality, fast implementations of data structures and algorithms. All implementations live in **public headers** that anyone can read.
- It is why you do not implement everything from scratch:
 - Dynamic array → `#include <vector>` and use `std::vector<>`.
 - Doubly-linked list → `#include <list>` and use `std::list<>`; no need to write your own `Node` class.
 - Sorting → `#include <algorithm>` and call `std::sort()`.
- Built for performance by some of the best programmers in the world, so you get an efficient, bug-free implementation.

Four primary components

1. **Containers** — store and retrieve data in different ways (vectors, lists, deques, stacks, queues, priority queues, ...).
 2. **Algorithms** — operate on data in containers (`std::sort()`, `std::min_element()`, `std::max_element()`, ...; mostly in `<algorithm>`).
 3. **Functions** — function objects, e.g. comparators like `std::less<>` and `std::greater<>`.
 4. **Iterators** — the interface between algorithms and containers.
- The STL also provides **utilities** (e.g. `std::swap()` in `<utility>`) and **memory allocators** (objects that handle memory management).

Remark

You are **not** required to memorize the STL. Even experienced programmers look things up. You *should* be comfortable with **iterators**, **pairs**, and **function objects**. If a concept was never mentioned in class, you are most likely not responsible for it (but it may still be useful).

11.2 Pairs and Tuples

11.2.1 ★ Pairs

- A **pair** groups two values and treats them as a single unit. The two objects may be of **different types**.
- Declared as `std::pair<T1, T2>`. Many containers rely on pairs internally (e.g. `std::map<>` and `std::unordered_map<>`, covered later).
- Accessed through the public members `first` and `second`. A pair is literally implemented as a templated struct with those two members.
- Pairs can be **copied**, **assigned**, **swapped**, and **compared** — so binding two types together does not force you to redefine basic operations.

Operation	Behavior
<code>std::pair<T1, T2> p;</code>	Default constructor: default-constructed values of T1 and T2
<code>std::pair<T1, T2> p(v1, v2);</code>	Creates a pair initialized to v1 and v2
<code>std::pair<T1, T2> p2(p1);</code>	Copy constructor: p2 becomes a copy of p1
<code>p2 = p1;</code>	Assignment operator
<code>p1 == p2;</code>	Equality: true only if both first and second match
<code>p1 < p2;</code>	Compares first; if those are equal, compares second. Works the same for >, <=, >=
<code>p.first;</code>	References the first element
<code>p.second;</code>	References the second element
<code>std::make_pair(v1, v2);</code>	Creates a pair with first = v1 and second = v2

```
std::pair<std::string, int32_t> myPair = std::make_pair("EECS", 281);
std::pair<std::string, int> myPair = {"EECS", 281}; // curly braces work too
```

- ★ **Comparison semantics:** compare first first; the second values only break ties.
 - {12,45} is greater than {10,953} because 12 > 10.
 - {10,45} is less than {10,953} because the first values tie, so 45 < 953 decides.

✗ Not required: Tuples (`std::tuple<>`)

A **tuple** does what a pair does, but with **any number** of elements. `#include <tuple>`. Pairs are enough for this class (you can also just use a struct with two members).

Operation	Behavior
<code>std::tuple<T1, T2, T3, ...> t;</code>	Default constructor: default-constructed component values
<code>std::tuple<T1, ...> t(v1, v2, v3, ...);</code>	Creates a tuple initialized with the given values
<code>std::tuple<T1, T2> t(p);</code>	Builds a two-element tuple from a pair p
<code>t2 = t1;</code>	Assignment operator
<code>t1 == t2;</code>	Compares first values, then second, then third, and so on
<code>std::make_tuple(v1, v2, v3, ...);</code>	Creates a tuple from the given arguments

```
std::tuple<std::string, int32_t, std::string, double> t =
    std::make_tuple("EECS", 281, "PI", 3.14);
std::get<0>(t) == "EECS";    std::get<1>(t) == 281;
std::get<2>(t) == "PI";      std::get<3>(t) == 3.14;
```

- `std::get<i>(t)` accesses component i (zero-indexed). The index must be known **at compile time**, so a runtime loop variable does not compile. Invalid indices are a compile error.
- `std::tuple_cat(t1, t2)` concatenates two tuples: a 3-element and a 4-element tuple produce a 7-element tuple.
- `std::tie(v1, v2, v3, v4) = t;` unpacks a tuple into individual variables. Use `std::ignore` in place of a name to skip a component.
- Because a pair is just a two-element tuple, `std::get<>`, `std::tuple_cat()`, and `std::tie()` also work on `std::pair<>`.

C++17 structured bindings (also optional, see the last section) make this cleaner:

```
std::pair<std::string, int> p = {"EECS", 281};
auto [str, num] = p; // str == "EECS", num == 281 (auto& [str, num] binds by reference)
```

11.3 Introduction to Iterators

11.3.1 Why Iterators Exist

- With a pure object-oriented design, each container is a class and each algorithm is a member function. With A algorithms and C containers you would write $A \times C$ implementations (one `find_min` for vector, one for list, one for deque, ...).
- That causes code duplication and bugs. We want to write each algorithm **once**.
- ★ **Iterators are the interface between containers and algorithms.** The algorithm never needs to know the container type or the memory layout; it only sees data values.



11.3.2 ★ Iterator Basics

- An **iterator** is an object used to traverse the elements of a container.
- Similar to pointers: can be dereferenced (`operator*`), incremented (`operator++`), compared (`operator==`), and assigned (`operator=`).
- **Better than pointers**: incrementing a pointer moves to the next memory address, but STL container elements are not guaranteed to be contiguous. Incrementing an **iterator** moves to the next *element*, contiguous or not.
- An iterator's internal behavior depends on its container (a vector iterator differs from a list iterator), but the **interface is the same** across containers. That is what lets one algorithm implementation serve many containers.

Getting iterators from a container

- `.begin()` returns an iterator to the **first** element.
- `.end()` returns an iterator **one past the last** element.
- `.cbegin()` / `.cend()` return **const** iterators; dereferencing one gives a reference to a const value that cannot be modified.
- **Exception: container adaptors** (stacks, queues, priority queues) do **not** support iteration.

```

std::vector<int32_t> vec = {1, 2, 3, 4};
for (std::vector<int32_t>::iterator it = vec.begin(); it != vec.end(); ++it) {
    std::cout << *it << " ";
} // for it

// same thing with a while loop
std::vector<int32_t>::iterator it = vec.begin();
while (it != vec.end()) {
    std::cout << *it++ << " "; // dereference first, THEN increment (postfix)
} // while
  
```

- You do not need to spell out the iterator type; `auto` deduces it.

Remark: `std::begin()` and `std::end()` (C++11)

These free functions do the same thing as the member functions but make generic code easier, and they also work on **C-style arrays**, which have no member `.begin()` / `.end()`.

```

int32_t arr[5] = {1, 2, 3, 4, 5};
auto start = std::begin(arr);
while (start != std::end(arr)) {
    std::cout << *start++ << " "; // prints 1 2 3 4 5
} // while
  
```

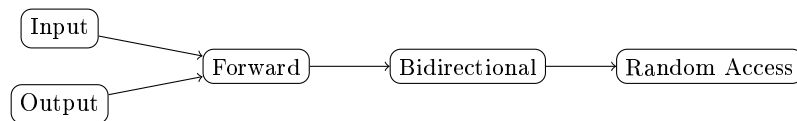
For any container `c` that supports iterators, `std::begin(c)` works instead of `c.begin()`. The member function form is more common by convention.

11.3.3 ★ Iterator Categories

- **Input iterator** — read only, forward moving, single pass.
 - Supports `*`, `++`, `==`, `!=`. No `-`.
 - No multiple passes: once incremented, the previous position is not guaranteed to be dereferenceable, even if you saved a copy.
 - Can be dereferenced multiple times, but only for the **current** element.
 - Comparisons are usually just to test whether the end of the range has been reached.
- **Output iterator** — write only, forward moving, single pass.
 - Can be written to (via `*`) and incremented (`++`), but **cannot be compared**. No `-`.
 - No multiple passes, and **no multiple writes** to the same element (after dereferencing and writing, you must increment before dereferencing again).
- **Forward iterator** — reads **and** writes, forward only.
 - Supports `*`, `++`, `==`, `!=`; supports **multiple passes** and multiple reads/writes to the same element. No `-`.
- **Bidirectional iterator** — everything a forward iterator can do, plus **backward movement** (`-`).
- **Random access iterator** — everything a bidirectional iterator can do, plus **pointer arithmetic** (`+`, `+=`, `-`, `-=`) and **pointer comparison** (`<`, `<=`, `>`, `>=`).

	Input	Output	Forward	Bidirectional	Random
Dereference (*) and read	✓		✓	✓	✓
Dereference (*) and write		✓	✓	✓	✓
Forward movement (++)	✓	✓	✓	✓	✓
Backward movement (-)				✓	✓
Multiple passes			✓	✓	✓
== and !=	✓		✓	✓	✓
Pointer arithmetic (+, -)					✓
Pointer comparison (<, >)					✓

★ Hierarchy



- Random access is **highest**; input/output are **lowest**. An iterator higher in the hierarchy supports everything below it.
- ★ Wherever an algorithm expects an iterator, you may pass any iterator that is **higher** on the hierarchy.
 - A random access iterator works anywhere a bidirectional, forward, input, or output iterator is expected.
 - A bidirectional iterator works in place of forward/input/output, but **not** in place of random access.

Container	Iterator Category
<code>std::vector<></code>	Random Access
<code>std::deque<></code>	Random Access
<code>std::string<></code>	Random Access
<code>std::list<></code>	Bidirectional
<code>std::set<></code> , <code>std::multiset<></code>	Bidirectional
<code>std::map<></code> , <code>std::multimap<></code>	Bidirectional
<code>std::unordered_set<></code> , <code>std::unordered_multiset<></code>	Forward
<code>std::unordered_map<></code> , <code>std::unordered_multimap<></code>	Forward
<code>std::forward_list<></code>	Forward

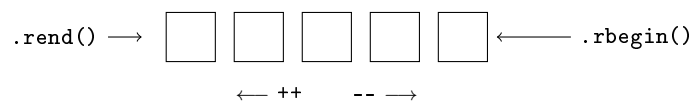
- ★ The category decides which algorithms are usable. `std::sort()` only accepts **random access** iterators, so it cannot sort a `std::list<>` — lists provide their own sort that uses bidirectional iterators.

11.4 Iterator Adaptors

Iterator adaptors build on the five iterator categories to add functionality.

11.4.1 ★ Reverse Iterators

- A **reverse iterator** traverses a container in reverse order. **Incrementing** a reverse iterator moves it **backward** in the container.
- Supported by every container with bidirectional or random access iterators.
- `.rbegin()` points to the **last** element (`std::rbegin()` also works).
- `.rend()` points **one before the first** element (`std::rend()` also works).
- `.crbegin()` / `.crend()` give **const** reverse iterators.



- ★ Passing reverse iterators into an algorithm is like handing it the range **backwards**: the algorithm runs normally but treats the last element as if it were the first. The result is the reverse of what normal iterators would produce.

```
std::vector<int32_t> vec = {1, 2, 3, 4, 5};
for (auto it = vec.rbegin(); it != vec.rend(); ++it) {
    std::cout << *it << " ";
    // prints 5 4 3 2 1
}
```

```

} // for it
// range constructor with reverse iterators -> holds contents in reverse order
std::vector<int32_t> vec_copy(vec.rbegin(), vec.rend()); // {5, 4, 3, 2, 1}
// iterators can copy data into a DIFFERENT container type as well
std::deque<int32_t> deq(vec.rbegin(), vec.rend()); // {5, 4, 3, 2, 1}
std::list<int32_t> lst(vec.rbegin(), vec.rend()); // {5, 4, 3, 2, 1}

```

✗ Not required: Stream iterators

Stream iterators traverse the contents of a **stream** as if it were a container.

- `std::istream_iterator<T>` is an **input** iterator that reads a stream (single pass only).
 - Constructed with a stream object → reads the first object in it. `std::istream_iterator<T> it{std::cin}` behaves like `std::cin » val`.
 - **Default constructed** → an **end-of-stream** iterator, comparable against another stream iterator to detect the end.
 - An object is read when the iterator is **incremented**, not when it is dereferenced. Dereferencing returns a copy of the most recently read object.

```

std::istream_iterator<int32_t> it{std::cin};
std::istream_iterator<int32_t> eof; // default constructed = end-of-stream
while (it != eof) { std::cout << *it++ << " "; }
// input "1 2 / 3 4 5 / 6 7 8 9" -> prints 1 2 3 4 5 6 7 8 9
// with std::string instead of int32_t, input "EECS 281 is fun" prints EECS 281 is fun

```

- `std::ostream_iterator<T>` is an **output** iterator that writes objects to a stream; it can take an optional **delimiter** that separates written elements.

```

std::vector<int32_t> vec = {1, 2, 3, 4, 5};
std::ostream_iterator<int32_t> out{std::cout};
std::copy(vec.begin(), vec.end(), out); // prints 12345

std::ostream_iterator<int32_t> out2{std::cout, " "}; // space delimiter
std::copy(vec.begin(), vec.end(), out2); // prints 1 2 3 4 5

// one-liner version
std::copy(vec.begin(), vec.end(), std::ostream_iterator<int32_t>{std::cout, " "});

```

✗ Not required: Insert iterators

By default output iterators work in **overwrite mode**. For example, `std::copy(first, last, result)` overwrites the data at `result`:

```

std::vector<int32_t> orig = {5, 6, 7, 8};
std::vector<int32_t> dest = {1, 2, 3, 4};
std::copy(orig.begin(), orig.end(), dest.begin()); // dest becomes {5, 6, 7, 8}

```

An **insert iterator** inserts instead of overwriting. Three kinds:

- **Back-insert iterator** (`std::back_inserter(c)`): assigning calls `.push_back()`. Requires a container with `.push_back()` (vector, deque, list).
- **Front-insert iterator** (`std::front_inserter(c)`): assigning calls `.push_front()`. Requires `.push_front()` (deque, list). Inserting a **range reverses** its order, since each element is pushed to the front in turn.
- **Generic insert iterator** (`std::inserter(c, pos)`): inserts anywhere; requires `.insert()` (most containers except container adaptors, arrays, and forward lists). **Incrementing it does nothing**; to insert elsewhere, redeclare it with the new position.

```

std::vector<int32_t> dest = {1, 2, 3, 4};
std::copy(orig.begin(), orig.end(), std::back_inserter(dest)); // {1,2,3,4,5,6,7,8}

std::deque<int32_t> d_dest = {1, 2, 3, 4};
std::copy(orig.begin(), orig.end(), std::front_inserter(d_dest)); // {8,7,6,5,1,2,3,4}

std::vector<int32_t> vec = {1, 2, 3, 6, 7, 8};
auto it = std::inserter(vec, vec.begin() + 3); // position of the 6
*it = 4; // {1, 2, 3, 4, 6, 7, 8}
*it = 5; // {1, 2, 3, 4, 5, 6, 7, 8}

```

X Not required: Move iterators

A **move iterator** yields an **rvalue reference** when dereferenced: for a move iterator `i`, `*i` gives `std::move(*i)` instead of `*i`. Useful when you want every value during iteration treated as an rvalue. Build one with `std::make_move_iterator()`.

```
std::vector<std::string> src = {"QWERTYUIOP", "ASDFGHJKL", "ZXCVBNM"};
std::vector<std::string> dst;
dst.insert(dst.end(), std::make_move_iterator(src.begin()),
           std::make_move_iterator(src.end()));
```

Each element of `src` is move-constructed into `dst`, leaving the originals in an unspecified state. Without move iterators the strings would be **copied**, which is inefficient for large objects that are not needed afterward.

X Not required: Auxiliary iterator functions (<iterator>)

Random access iterators support arithmetic (`it += 10;`) and subtraction (`it_last - it_first`). Other iterator categories do not, so these helpers exist:

- `std::advance(it, n)` — moves `it` forward by `n` positions (equivalent to `it += n`). `n` may be negative only for bidirectional/random access iterators.
- `std::distance(first, last)` — number of elements between the two iterators. `first` should come before `last`.
- `std::next(it, n)` — returns an iterator `n` positions ahead of `it`.
- `std::prev(it, n)` — returns an iterator `n` positions behind `it`.
- Unlike `std::advance()`, `std::next()` and `std::prev()` do **not** move the iterator passed in.

```
std::list<int32_t> lst = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
auto it = std::begin(lst);      // 'it' points to 0
std::advance(it, 3);            // 'it' points to 3
std::advance(it, 4);            // 'it' points to 7
std::advance(it, -2);           // 'it' points to 5
auto foo = std::next(it, 4);     // 'foo' -> 9, 'it' still 5
auto bar = std::prev(it, 4);     // 'bar' -> 1, 'it' still 5
std::cout << std::distance(bar, foo) << '\n'; // prints 8
```

(The parameter `n` is technically a **difference_type**, a numeric type able to represent iterator distance across compilers. Treat it like an `int`.)

11.5 Predicates in the STL

- ★ A **function object (functor)** is an object that can be used with `operator()` to behave like a function.
- A **predicate** is a function object that returns a `bool` and is **stateless** (same input always gives the same result).
 - **Unary predicate**: takes one argument, checks a property of it.
 - **Binary predicate**: takes two arguments, compares a property of them.
- A **comparator** is a binary predicate that returns whether one object goes before or after another.

★ `std::less<>` and `std::greater<>`

```
std::less<int32_t> comp;          // true if first argument < second (uses operator<)
std::cout << comp(4, 5) << '\n';  // 1 (true)
std::cout << comp(5, 4) << '\n';  // 0 (false)
std::cout << comp(5, 5) << '\n';  // 0 (false)

std::greater<int32_t> comp2;      // true if first argument > second (uses operator>)
std::cout << comp2(4, 5) << '\n';  // 0 (false)
std::cout << comp2(5, 4) << '\n';  // 1 (true)
std::cout << comp2(5, 5) << '\n';  // 0 (false)
```

- The element type `T` must implement `operator<` for `std::less<T>` and `operator>` for `std::greater<T>`. To use them on a custom struct/class, define the corresponding operator.
- Other predefined function objects: `std::equal_to<>`, `std::not_equal_to<>`, `std::less_equal<>`, `std::greater_equal<>`. The list is not comprehensive, but `std::less<>` and `std::greater<>` are by far the most common (e.g. in `std::priority_queue<>` declarations).

11.6 The Algorithm Library

11.6.1 How STL Algorithms Work

- Algorithms use iterators to reach a container's data, so they see **only the data values**, not how the data is organized in memory.

Important

STL algorithms can modify the data values of a container, but they cannot modify the structure of the container. Iterators expose data values, not the container type. This matters most for algorithms that look like they mutate the container, such as `std::remove()` and `std::remove_if()`: they only reorder data to make removal easier. The actual removal must be done separately through a container member function such as `.erase()`.

- ★ Algorithms take an **iterator range**: a **begin** iterator and an **end** iterator one past the last element, giving access to `[begin, end)`. **begin** is **included**; **end** is **excluded**.
- Any subsection of a container can be captured as a range and passed to an algorithm.

11.6.2 ★ Minimum and Maximum Algorithms

Function	Behavior
<code>std::min(a, b, comp)</code>	Returns the smaller of a and b . Comparator optional (operator< by default)
<code>std::max(a, b, comp)</code>	Returns the larger of a and b
<code>std::minmax(a, b, comp)</code>	Returns a pair : smaller as first , larger as second . If equivalent, returns <code>std::make_pair(a, b)</code>
<code>std::min_element(first, last, comp)</code>	Returns an iterator to the smallest element in <code>[first, last)</code>
<code>std::max_element(first, last, comp)</code>	Returns an iterator to the largest element in <code>[first, last)</code>
<code>std::minmax_element(first, last, comp)</code>	Returns a pair of iterators (smallest, largest)

```
std::cout << std::min(14, 12) << '\n'; // prints 12
std::cout << std::max(14, 12) << '\n'; // prints 14
std::pair<int32_t, int32_t> p = std::minmax(14, 12); // p.first = 12, p.second = 14

// more than two values -> pass an initializer list
int32_t a = 12, b = 15, c = 11, d = 19, e = 10, f = 14;
std::cout << std::min({a, b, c, d, e, f}) << '\n'; // prints 10
std::cout << std::max({a, b, c, d, e, f}) << '\n'; // prints 19

// entire container -> use the _element versions (they return ITERATORS)
std::vector<int32_t> vec = {6, 2, 4, 9, 8, 3, 7, 1, 5};
std::cout << *std::min_element(vec.begin(), vec.end()) << '\n'; // prints 1
std::cout << *std::max_element(vec.begin(), vec.end()) << '\n'; // prints 9
auto it_pair = std::minmax_element(vec.begin(), vec.end());
std::cout << *it_pair.first << " " << *it_pair.second << '\n'; // prints 1 9
```

- Complexity**: linear in the number of elements compared. Exception: `std::min()` / `std::max()` with only **two** arguments are constant time.

11.6.3 ★ Sorting Algorithms

Function	Behavior
<code>std::sort(first, last, comp)</code>	Sorts <code>[first, last)</code> in ascending order, or by comp . Relative order of equivalent elements is not guaranteed
<code>std::partial_sort(first, middle, last, comp)</code>	Rearranges <code>[first, last)</code> so that <code>[first, middle)</code> holds exactly what it would hold if the whole range were sorted
<code>std::stable_sort(first, last, comp)</code>	Same as sort , but preserves the relative order of equivalent elements (if a == b and a came first, a still comes first)

- ★ **Comparator rule for sorting**: for two elements passed into the comparator, **true** means the first element comes **before** the second in sorted order; otherwise it comes after.
 - Descending order $\Rightarrow$ pass `std::greater<>`, since `comp(a,b)` is true only when $a > b$.

- ★ `std::sort()` requires **random access iterators** (the implementation uses iterator arithmetic), so it cannot sort a container without them (e.g. `std::list<>`).
- **Complexity** of `std::sort()`: $\Theta(n \log(n))$ (why is explained in Chapter 14).

```
std::vector<int32_t> vec = {6, 2, 4, 9, 8, 3, 7, 1, 5};
std::sort(vec.begin(), vec.end());           // 1 2 3 4 5 6 7 8 9
std::sort(vec.begin(), vec.end(), std::greater<int32_t>()); // 9 8 7 6 5 4 3 2 1

// sorting a SUBRANGE: only 4, 9, 8, 3 are sorted relative to each other
std::vector<int32_t> v = {6, 2, 4, 9, 8, 3, 7, 1, 5};
std::sort(v.begin() + 2, v.begin() + 6);      // 6 2 3 4 8 9 7 1 5

// partial_sort: guarantee only the smallest three positions
std::vector<int32_t> pv = {4, 2, 7, 5, 3, 6};
std::partial_sort(pv.begin(), pv.begin() + 3, pv.end()); // 2 3 4 7 5 6
```

- ★ **Sorting with reverse iterators**: the algorithm sees the data **in reverse**, so the final result is flipped. Sorting a reverse range "ascending" leaves the container in **descending** order.

```
std::vector<int32_t> u = {4, 2, 7, 8, 3, 6};
std::vector<int32_t> v = {4, 2, 7, 8, 3, 6};
std::sort(u.begin(), u.end());           // 2 3 4 6 7 8
std::sort(v.rbegin(), v.rend());         // 8 7 6 4 3 2

std::vector<int32_t> a = {4, 2, 7, 8, 3, 6};
std::vector<int32_t> b = {4, 2, 7, 8, 3, 6};
std::sort(a.begin() + 2, a.begin() + 5); // 4 2 3 7 8 6
std::sort(b.rbegin() + 1, b.rbegin() + 4); // 4 2 8 7 3 6
```

11.6.4 ★ Find Algorithms

Function	Behavior
<code>std::find(first, last, val)</code>	Returns an iterator to the first element in <code>[first, last)</code> equal to <code>val</code> (uses <code>operator==</code>). Returns <code>last</code> if not found
<code>std::find_if(first, last, pred)</code>	Returns an iterator to the first element for which the unary predicate <code>pred</code> returns <code>true</code> . Returns <code>last</code> if none

- **Complexity**: linear in the distance between `first` and `last`, since every element may be compared.

```
struct IsOddPred {
    bool operator() (int32_t i) { return i % 2 == 1; } // operator()()
};

std::vector<int32_t> vec = {24, 66, 12, 74, 26, 43, 92, 95, 92, 71};
auto it1 = std::find(vec.begin(), vec.end(), 92); // first 92 (between 43 and 95)
auto it2 = std::find(vec.begin(), vec.end(), 93); // vec.end(), 93 is not present
auto it3 = std::find_if(vec.begin(), vec.end(), IsOddPred()); // 43, first odd number
```

11.6.5 ★ Removal Algorithms and the Erase-Remove Idiom

Function	Behavior
<code>std::remove(first, last, val)</code>	"Removes" all elements equal to <code>val</code> ; returns an iterator to the new end of the range
<code>std::remove_if(first, last, pred)</code>	"Removes" all elements for which <code>pred</code> returns <code>true</code> ; returns an iterator to the new end
<code>std::remove_copy(first, last, dest, val)</code>	Copies <code>[first, last)</code> to <code>dest</code> , except elements equal to <code>val</code>
<code>std::remove_copy_if(first, last, dest, pred)</code>	Copies <code>[first, last)</code> to <code>dest</code> , except elements satisfying <code>pred</code>

- ★ **The catch**: `std::remove()` does **not** physically remove anything. It moves the elements that should be removed toward the **back** and returns an iterator one past the last element that should be kept. Everything after that iterator is junk data.
- ★ **Erase-remove idiom**: always pair the algorithm with `.erase()` on the container.

```
auto it = std::remove(vec.begin(), vec.end(), 1);
vec.erase(it, vec.end());
```

```
// or, in one line:
vec.erase(std::remove(vec.begin(), vec.end(), 1), vec.end());
vec.erase(std::remove_if(vec.begin(), vec.end(), IsOddPred()), vec.end());

std::vector<int32_t> v1 = {1, 3, 2, 1, 4, 1, 1, 3};
v1.erase(std::remove(v1.begin(), v1.end(), 1), v1.end()); // v1 = {3, 2, 4, 3}

std::vector<int32_t> v2 = {1, 3, 2, 1, 4, 1, 1, 3};
v2.erase(std::remove_if(v2.begin(), v2.end(), IsOddPred()), v2.end()); // v2 = {2, 4}

// copy versions leave the original container unchanged
std::vector<int32_t> v3 = {1, 3, 2, 1, 4, 1, 1, 3};
std::vector<int32_t> v4(v3.size()); // must be big enough
std::remove_copy(v3.begin(), v3.end(), v4.begin(), 1); // v4 = {3, 2, 4, 3, 0, 0, 0, 0}
v4.erase(std::remove(v4.begin(), v4.end(), 0), v4.end()); // v4 = {3, 2, 4, 3}
```

- **Complexity:** all removal algorithms are linear in the distance between **first** and **last**.

Remark: C++20 `std::erase_if()`

Starting in C++20, certain containers gain `std::erase_if()`, which **physically** erases elements and returns how many were erased.

```
std::vector<int32_t> v = {1, 3, 2, 1, 4, 1, 1, 3};
auto num_values_erased = std::erase_if(v, IsOddPred()); // v = {2, 4}
std::cout << num_values_erased << '\n'; // prints 6
```

11.6.6 ★ Lower and Upper Bound Algorithms

Function	Behavior
<code>std::lower_bound(first, last, val, comp)</code>	Iterator to the first element that does not compare less than val . The value there may equal val
<code>std::upper_bound(first, last, val, comp)</code>	Iterator to the first element that compares strictly greater than val
<code>std::equal_range(first, last, val, comp)</code>	Pair of iterators: first = lower bound, second = upper bound. Zero-length subrange if val is absent

- If no element fits, all of these return **last**.
- ★ The data **must be sorted**. On unsorted data the behavior is **undefined**.
- ★ If **val** exists, `[lower_bound, upper_bound)` is exactly the range of elements equal to **val** (assuming no custom comparator).
- If **val** is absent, `equal_range()` returns two iterators pointing to the closest value greater than **val**, or **last** if **val** is greater than everything.
- **Complexity:** $\Theta(\log(n))$ for a sorted container with random access iterators (binary search).

1	2	3	3	5	7	9	9
---	---	---	---	---	---	---	---

`lower_bound(..., 3) → the first 3.` `upper_bound(..., 3) → the element just past the last 3 (the 5).`

```
std::vector<int32_t> vec = {1, 2, 5, 5, 6, 9};
auto it1 = std::lower_bound(vec.begin(), vec.end(), 5); // first 5
auto it2 = std::upper_bound(vec.begin(), vec.end(), 5); // 6
auto it3 = std::lower_bound(vec.begin(), vec.end(), 7); // 9
auto it4 = std::upper_bound(vec.begin(), vec.end(), 7); // 9
auto it5 = std::lower_bound(vec.begin(), vec.end(), 10); // vec.end()
auto it6 = std::upper_bound(vec.begin(), vec.end(), 10); // vec.end()
auto pr1 = std::equal_range(vec.begin(), vec.end(), 5); // {first 5, 6}
auto pr2 = std::equal_range(vec.begin(), vec.end(), 0); // {1, 1}
```

11.6.7 Other Algorithms [reference only]

These are worth knowing about, but you do not need to memorize them.

Replacement algorithms [reference only]

Function	Behavior
<code>std::replace(first, last, old_val, new_val)</code>	Replaces all instances of <code>old_val</code> with <code>new_val</code>
<code>std::replace_if(first, last, pred, new_val)</code>	Replaces all elements satisfying <code>pred</code> with <code>new_val</code>
<code>std::replace_copy(first, last, dest, old_val, new_val)</code>	Copies to <code>dest</code> , replacing <code>old_val</code> with <code>new_val</code>
<code>std::replace_copy_if(first, last, dest, pred, new_val)</code>	Copies to <code>dest</code> , replacing values satisfying <code>pred</code>

- All are linear in the distance between `first` and `last`.

```
std::vector<int32_t> vec = {1, 3, 2, 1, 1, 7, 3, 6, 4, 5};
std::replace(vec.begin(), vec.end(), 1, 0);           // {0,3,2,0,0,7,3,6,4,5}
std::replace_if(vec.begin(), vec.end(), IsOddPred(), 8); // {0,8,2,0,0,8,8,6,4,8}

std::vector<int32_t> foo(vec.size());
std::replace_copy(vec.begin(), vec.end(), foo.begin(), 8, 9); // foo={0,9,2,0,0,9,9,6,4,9}
std::vector<int32_t> bar(foo.size());
std::replace_copy_if(foo.begin(), foo.end(), bar.begin(), IsOddPred(), 4);
// bar = {0,4,2,0,0,4,4,6,4,4}; vec and foo are unchanged by the _copy versions
```

Transformation algorithms [reference only]

- `std::transform(first1, last1, result, op)` — applies the **unary** operation `op` to every element in `[first1, last1)` and writes each result starting at `result`. Returns an iterator one past the last element written.
- `std::transform(first1, last1, first2, result, binary_op)` — applies a **binary** operation, taking each element of the first range as the first argument and the matching element of the second range as the second argument.
- The caller must ensure the destination range is large enough (resize it, or use insert iterators). To overwrite the original range, pass the same iterator for `first1` and `result`.

```
std::string s = "eecs281";
std::transform(s.begin(), s.end(), s.begin(), ::toupper);
std::cout << s << '\n';           // prints "EECS281"

int32_t binary_op(int32_t a, int32_t b) { return a * a + b; }
std::vector<int32_t> v1 = {5, 6, 7, 8, 9};
std::vector<int32_t> v2 = {1, 2, 3, 4, 5};
std::vector<int32_t> v3;           // stores v1 * v1 + v2
std::transform(v1.begin(), v1.end(), v2.begin(), std::back_inserter(v3), binary_op);
// v3 = {26, 38, 52, 68, 86}
```

- Note on `::toupper`: with using namespace `std`, the two leading colons tell the program to use the C library version, not `std::toupper()` from `<locale>`.

Copy algorithms [reference only]

- `std::copy(first, last, dest)` — copies `[first, last)` into the range starting at `dest`; returns an iterator one past the last element copied.
- `std::copy_if(first, last, dest, pred)` — copies only elements satisfying `pred`; the returned iterator lets you resize the destination afterward or keep writing from where you left off.
- `dest` must **not** point into `[first, last)` (overlap problems). Both are linear in the range size.

```
std::vector<int32_t> vec = {0, 1, 2, 3, 4, 5, 6, 7};
std::deque<int32_t> deq(4);
std::copy(vec.begin() + 2, vec.begin() + 6, deq.begin()); // deq = {2, 3, 4, 5}

std::vector<int32_t> v = {4, 2, 7, 5, 3, 6};
std::vector<int32_t> odds(v.size());
auto end = std::copy_if(v.begin(), v.end(), odds.begin(), IsOddPred());
// odds = {7, 5, 3, 0, 0, 0}
odds.resize(std::distance(odds.begin(), end));           // odds = {7, 5, 3}
```

Reversal algorithms [reference only]

- `std::reverse(first, last)` — reverses the order of elements in the range.
- `std::reverse_copy(first, last, dest)` — writes the range to `dest` in reverse order, leaving the original alone.

```
std::vector<int32_t> vec = {1, 2, 6, 3, 8, 9, 4, 5, 7};
std::reverse(vec.begin(), vec.end());           // {7,5,4,9,8,3,6,2,1}
std::reverse(vec.begin() + 3, vec.begin() + 6); // {7,5,4,3,8,9,6,2,1}
```

`std::nth_element()` [reference only]

- `std::nth_element(first, nth, last, comp)` rearranges `[first, last)` so that the element at `nth` is the one that **would** be there if the range were fully sorted.
- Requires **random access** iterators. **Complexity:** $\Theta(n)$, versus $\Theta(n \log(n))$ for a full sort — useful when you only care about one position (e.g. the 281st smallest of 1,000,000 values, or the median).
- The rest of the range is left in **arbitrary order**. The only guarantee: nothing before `nth` is greater, and nothing after it is smaller.

```
std::vector<int32_t> vec = {14, 33, 74, 11, 25, 75, 24, 47, 85, 17, 59};
size_t middle_idx = vec.size() / 2;
std::nth_element(vec.begin(), vec.begin() + middle_idx, vec.end());
std::cout << vec[middle_idx] << '\n'; // prints 33, the median
```

11.6.8 Algorithms Covered Elsewhere

- **Chapter 13:** `std::set_union()`, `std::set_intersection()`, `std::set_difference()`, `std::set_symmetric_difference()`.
- **Chapter 16:** `std::equal()` and `std::unique()`.
- The `<algorithm>` library is much larger than this. <https://cppreference.com/> is the standard reference for up-to-date features.

11.7 Standard Math Functions and Numeric Limits

X Not required: the math library, precision, and numeric limits

`<cmath>` provides common math operations (not a comprehensive list):

Function	Behavior
<code>std::exp(x)</code>	Returns e^x
<code>std::log(x)</code>	Returns $\ln(x)$
<code>std::log10(x)</code> / <code>std::log2(x)</code>	Returns $\log_{10}(x)$ / $\log_2(x)$
<code>std::pow(base, exponent)</code>	Returns $\text{base}^{\text{exponent}}$
<code>std::sqrt(x)</code> / <code>std::cbrt(x)</code>	Square root / cube root of x
<code>std::ceil(x)</code>	Smallest integer value not less than x
<code>std::floor(x)</code>	Largest integer value not greater than x
<code>std::round(x)</code>	Nearest integer to x ; halfway cases round away from zero
<code>std::abs(x)</code>	Absolute value (may be unsafe for floating point)
<code>std::fabs(x)</code>	Absolute value, safer for <code>double</code> and <code>float</code>

- **Warning:** the default `std::abs()` is the C version that takes only integers. For floating point you must `#include <cmath>`; using `std::fabs()` is safer since it states your intent and prevents undetectable truncation.
- These are general-purpose, so their versatility makes them slower for simple work. `std::pow(3, 20.65)` is great; for 3^2 , plain `3*3` is much faster than `std::pow(3, 2)`.

`std::setprecision(n)` (in `<iomanip>`) sets floating point output to `n` digits:

```
double pi = 3.14159265358979;
std::cout << std::setprecision(1) << pi << '\n'; // 3
std::cout << std::setprecision(3) << pi << '\n'; // 3.14
std::cout << std::setprecision(6) << pi << '\n'; // 3.14159
std::cout << std::setprecision(12) << pi << '\n'; // 3.141592653589
```

`std::numeric_limits<>` (in `<limits>`) retrieves properties of arithmetic types (`bool`, `char`, `int`, `float`, `double`, and variants):

Function	Behavior
<code>std::numeric_limits<T>::min()</code>	Minimum finite value. For non-floating-point types this is the lowest value. For floating point types with denormalization it is the smallest positive finite value
<code>std::numeric_limits<T>::lowest()</code>	The lowest finite value; nothing of type T can be lower
<code>std::numeric_limits<T>::max()</code>	The maximum finite value
<code>std::numeric_limits<T>::infinity()</code>	A special infinity value if T supports it (only useful for <code>float/double</code>); otherwise returns 0

- `numeric_limits<int>::min() == -2147483648` `numeric_limits<double>::min() == 2.22507E-308` (positive but tiny!)
- `numeric_limits<int>::lowest() == -2147483648` `numeric_limits<double>::lowest() == -1.79769E308`
- `numeric_limits<int>::max() == 2147483647` `numeric_limits<double>::max() == 1.79769E308`
- Infinity is **not defined for integers**; use `std::numeric_limits<double>::infinity()`. It is considered larger than 1.79769E308.
- `INT_MIN / INT_MAX` in `<climits>` equal `numeric_limits<int>::min() / max()`, but the numeric limits style is preferred in C++.

`std::iota(first, last, val)` (in `<numeric>`) fills `[first, last)` with sequentially increasing values starting at `val`. T must be incrementable. Useful for the union-find algorithm in Chapter 13.

```
std::vector<int32_t> vec(10);
std::iota(vec.begin(), vec.end(), 1);           // {1,2,3,4,5,6,7,8,9,10}
std::iota(vec.begin() + 3, vec.begin() + 7, 11); // {1,2,3,11,12,13,14,8,9,10}
```

`std::accumulate(first, last, init, op)` (in `<numeric>`) sums the range and adds the result to `init`. An optional binary operation replaces summation.

```
std::vector<int32_t> vec = {1, 2, 3, 4, 5};
std::cout << std::accumulate(vec.begin(), vec.end(), 0) << '\n'; // prints 15
std::cout << std::accumulate(vec.begin(), vec.end(), 5) << '\n'; // prints 20

int32_t prod(int32_t accumulator, int32_t element) { return accumulator * element; }
std::cout << std::accumulate(vec.begin(), vec.end(), 1, prod) << '\n'; // prints 120
```

- The binary operation takes an **accumulator** (starting at `init`) and an **element**; the call above behaves exactly like `for (int32_t elt : vec) init = prod(init, elt);`
- This makes `accumulate` work on custom objects that do not define `operator+`, e.g. summing a `wealth` member across a vector of `Students`.
- The return type is the type of the third argument `init`.** Passing 0.00 returns a `double`; passing 0 returns an `int`. (The extra zeros are not needed — 0. is enough to indicate floating point.)

11.8 Lambdas and the Functional Library

✗ Not required (but a very useful skill): Lambdas

Motivation: defining a whole named functor for one-time use is excessive. A **lambda** (C++11) builds a short-lived, anonymous function object **in place**, right where it is used.

```
// instead of defining struct IsOddPred { ... }; and passing IsOddPred()
std::vector<int32_t> vec = {24, 66, 12, 74, 26, 43, 92, 95, 92, 71};
auto it = std::find_if(vec.begin(), vec.end(), [](int32_t i) {
    return i % 2 == 1;
});
```

Anatomy: `[capture clause] (parameter list) function body`

- Capture clause:** square brackets; captures variables from the surrounding scope.
- Parameter list:** the arguments the function would normally take.
- Function body:** the behavior, between curly braces.
- A lambda can be called directly, or stored in a variable and used like a functor:

```
std::cout << [](int32_t i){ return i % 2 == 1; }(5) << '\n'; // prints 1 (true)
auto is_odd_pred = [](int32_t i){ return i % 2 == 1; };
```

```
std::cout << is_odd_pred(6) << '\n'; // prints 0 (false)
```

- Like a normal function, a lambda need not take parameters: `[]() { std::cout << "hello world" << '\n'; }`
- **Type of a lambda:** unspecified, and every lambda instance has its own unique unnamed type. Use `auto`. To pass one into a function, prefer a **templated function** over `std::function<>` (the latter adds indirection and overhead, and its type must change if the lambda's signature changes).

Return type: usually deduced. To force a different one, use the arrow operator: `[capture] (params) -> return_type body`

```
[](double x, double y) { return x + y; }(1.23, 2.46); // 3.69
>[](double x, double y) -> int32_t { return x + y; }(1.23, 2.46); // 3 (truncated)
```

Variable capture: a lambda can use variables from the enclosing scope if it captures them.

```
int32_t factor; std::cin >> factor;
std::find_if(vec.begin(), vec.end(), [factor](int32_t i) { return i % factor == 0; });
```

- **By value:** the lambda stores a read-only **copy** taken at capture time. Later changes outside do not affect it. (Add `mutable` right after the parameter list to modify a by-value capture inside the lambda.)
- **By reference (&):** changes outside are visible inside, and changes inside are visible outside. **Warning:** the captured variables must not go out of scope before the lambda is used, or you get undefined behavior.

```
int32_t var = 281;
auto func = [var]() { std::cout << var << '\n'; };
++var; // var is now 282
func(); // prints 281 (lambda holds its own copy)

int32_t var2 = 281;
auto func2 = [&var2]() { std::cout << var2 << '\n'; };
++var2; // var2 is now 282
func2(); // prints 282 (lambda holds a reference)

auto func3 = [&var2]() { var2 += 10; };
func3(); // var2 becomes 292 -- changes inside are visible outside
```

Capture clause options

- `[]` captures nothing. `[=]` captures all used locals **by value**. `[&]` captures all used locals **by reference**.
- `[foo]` only `foo` by value. `[&foo]` only `foo` by reference. `[foo, bar]` both by value.
- `[=, &foo]` all by value except `foo` by reference. `[&, foo]` all by reference except `foo` by value.
- `[this]` captures a reference to the current object (useful inside a member function when the lambda needs member variables).
- **Style:** capturing everything (`[=]` or `[&]`) is generally bad practice; list the variables you actually need.

Lambdas with the STL: `std::for_each(first, last, fn)` applies `fn` to every element in a range (unlike a range-for loop, it works on a **subrange**) and returns `fn`.

```
std::vector<int32_t> vec = {1, 2, 3, 4, 5, 6, 7, 8};
std::for_each(vec.begin(), vec.end(), [](int32_t& i) { i += 281; });
// vec = {282, 283, ..., 289} -- take the parameter BY REFERENCE to modify elements

// define a comparator inline instead of writing a separate struct
std::sort(vec2.begin(), vec2.end(), [](const Student& lhs, const Student& rhs) {
    return lhs.age < rhs.age;
});
std::for_each(vec2.begin(), vec2.end(), [](const Student& s) { std::cout << s.name << ' '; });

// copy_if with a back-insert iterator: no need to pre-allocate the result
std::copy_if(nums.begin(), nums.end(), std::back_inserter(result),
    [](int32_t i) { return i % n == 0; });
```

Generic lambdas (C++14): use `auto` in the parameter list to accept arbitrary types. The compiler creates an anonymous templated function object. Not every parameter has to be generic.

```
auto add_two_values = [](auto x, auto y) { return x + y; };
add_two_values(1, 2); // 3
add_two_values(1, 2.5); // 3.5
add_two_values(std::string{"eecs"}, std::string{"281"}); // "eecs281"
```

Templated lambdas (C++20): the template parameter goes between the capture clause and parameter list: `[]<typename T>(T first, T second) { return first + second; }`

✗ Not required: `std::function<>` and `std::bind()` (<functional>)

`std::function<>` is a general-purpose polymorphic **function wrapper** that can store any callable object: named functions, functors, or lambdas. Syntax: `std::function<return_type(parameter_types)>`.

```
bool func(const std::string& str, int32_t val);
std::function<bool(const std::string&, int32_t)> my_func_object = func;

// a lambda with a matching signature also works
std::function<bool(const std::string&, int32_t)> f2 =
    [](const std::string& str, int32_t val) { return str == "eecs" && val == 281; };

// DOES NOT COMPILE: cannot pass a const std::string& into a std::string& parameter
std::function<bool(const std::string&, int32_t)> f3 =
    [](std::string& str, int32_t val) { return str == "eecs" && val == 281; };
```

- Any callable that *can be called* with the specified return and parameter types can be assigned (e.g. a lambda taking `const std::string&` can be stored in a `std::function<bool(std::string, int32_t)>`).
- This lets you hold callables of different underlying types in one container, or store one as internal state in a class.
- Use it only when you need it:** `std::function<>` involves inheritance, virtual functions, and dynamic memory allocation. Prefer a simpler representation when possible.

```
std::vector<std::function<void()>> callables;
callables.push_back(foo);           // regular function
callables.push_back(std::bind(bar, 281)); // bound function
callables.push_back(Baz());         // functor
callables.push_back([]() { std::cout << "fun!\n"; }); // lambda
for (auto& f : callables) { f(); }  // prints "EECS 281 is fun!"
```

`std::bind(callable, args...)` binds parameters of a function to fixed values, returning a function object whose `operator()` takes the remaining unbound arguments.

```
void print(int32_t num) { std::cout << num << '\n'; }
auto func = std::bind(print, 281);
func();    // prints 281
```

- `std::bind()` **copies** the arguments passed in. To track later changes, wrap the argument in `std::ref()`.
- `std::placeholders::_1, _2, ...` leave arguments unbound; the number says which call argument fills each slot. Placeholders can also **reorder** parameters.
- Because it returns a function object, the result can be passed into STL algorithms.

```
int32_t val = 280;
auto f1 = std::bind(print, val);
auto f2 = std::bind(print, std::ref(val)); // reference wrapper
++val; // val becomes 281
f1();  // still prints 280 (copy)
f2();  // prints 281 (reference)

void print2(const std::string& department, int32_t num);
auto g = std::bind(print2, "EECS", std::placeholders::_1);
g(281); // prints "EECS 281"
auto h = std::bind(print2, std::placeholders::_2, std::placeholders::_1);
h(281, "EECS"); // prints "EECS 281" (arguments swapped)
std::for_each(classes.begin(), classes.end(),
    std::bind(print2, "EECS", std::placeholders::_1));
```

- In C++14 and later, **lambdas do the same thing more cleanly**: `auto func = [&department](int32_t num) { print2(department, num); };`

11.9 Random Number Generators

✗ Not required (but valuable for generating test files): randomization

`rand()` is the standard quick way to generate a random integer. Use modulo to bound the range, and add an offset for a non-zero start:

```
int32_t r1 = rand() % 100; // random number 0-99
```

```
int32_t r2 = rand() % 100 + 1;    // random number 1-100
int32_t r3 = rand() % 50 + 281;  // random number 281-330
```

- Given the same **seed**, `rand()` always produces the same sequence. Without an explicit seed, the seed defaults to 1.
- Internally it applies math to the seed to get a number that looks unrelated, then applies the same math to that result for the next number, and so on until the seed is reset.
- `srand(n)` sets the seed to `n`; all later `rand()` calls follow that seed's sequence. Resetting to the original seed reproduces the original sequence.
- **Downsides:** `rand()` is not great statistically, and using modulo **biases** the output, because remainders are not uniformly distributed (restricted to 0–1999, 500 may appear more often than 1500). Prefer `<random>`.

The `<random>` library relies on two kinds of objects:

- **Generators:** function objects producing uniformly distributed numbers.
- **Distributions:** transform a generator's numbers to follow a specific distribution (Uniform, Normal, Binomial, Poisson, Exponential, Geometric, ...).
- A generator should be paired with a distribution. `std::mt19937` (Mersenne Twister) is the main general-purpose pseudorandom generator here and is much better than `rand()`.

```
// seed from the system clock -> different every run
auto seed = std::chrono::high_resolution_clock::now().time_since_epoch().count();
std::mt19937 gen(seed);
std::cout << gen() << '\n';

// or seed from a std::random_device ("true" randomness from hardware/stochastic sources;
// expensive, so it is usually only used to seed the generator)
std::random_device rd;
std::mt19937 gen2(rd());
```

- To bound the output, use a **distribution**, not modulo. `std::uniform_int_distribution<>` for integer types, `std::uniform_real_distribution<>` for floating point. Both take bounds `a` and `b`; every value in `[a, b]` is equally likely (a **uniform** distribution).
- Apply the distribution to the generator by passing the generator into the distribution with `operator()`.

```
std::random_device rd;
std::mt19937 gen(rd());
std::uniform_int_distribution<int32_t> dist(1, 100);
for (int32_t i = 0; i < 10; ++i) { std::cout << dist(gen) << " "; }
// e.g. 60 93 40 96 91 22 35 90 60 21    (different every run)

std::uniform_real_distribution<double> rdist(1, 100); // floating point version
```

Other uses: `std::shuffle(first, last, g)` in `<algorithm>` rearranges `[first, last)` randomly, swapping each element with another random element picked using `g()`.

```
std::vector<int32_t> v = {1, 2, 3, ..., 15};
std::random_device rd; std::mt19937 gen(rd());
std::shuffle(v.begin(), v.end(), gen);
```

Non-numeric data: to generate random words, read a dictionary file into a vector and use a distribution over the valid indices.

```
std::ifstream fin; fin.open("dictionary.txt");
std::vector<std::string> words; std::string word;
while (fin >> word) { words.push_back(word); }
fin.close();

std::random_device rd; std::mt19937 gen(rd());
std::uniform_int_distribution<int32_t> random_word_dist(0, words.size() - 1);
std::string random_word = words[random_word_dist(gen)];
```

Remark: `rand()` and the `<random>` distributions are **not guaranteed to be portable**. The same code, same seed, and same everything may produce different numbers on a different platform or compiler, so be careful when sharing files that depend on randomization.

11.10 Additional C++ Standard Library Features

✗ Not required: C++17 and beyond

C++ is still actively updated. C++11 (2011) added `auto`, `nullptr`, move semantics, lambdas, and more; C++14 added return type deduction and generic lambdas. These are good to know even if you cannot use them yet.

Structured bindings (C++17) — bind names to the subobjects of another object.

Syntax: `<type> [<name>, <name>, ...] = <tuple-like object>`, where `<type>` is usually `auto` (by value, reference, or const reference).

```
std::pair<std::string, int> p = {"EECS", 281};
auto [str, num] = p;           // str == "EECS", num == 281
const auto& [s2, n2] = p;     // const reference binding

std::tuple<std::string, int, char> t = {"EECS", 281, 'A'};
auto [str3, num3, grade] = t;

int32_t arr[3] = {203, 280, 281}; // also works with std::array
auto [x, y, z] = arr;           // 203, 280, 281 (NOT possible with a vector,
                                // whose structure is unknown at compile time)

struct ClassName { std::string department = "EECS"; int32_t number = 281; };
ClassName cn; auto [a, b] = cn; // "EECS", 281
```

Structured bindings give a clean way to emulate **multiple return values**. The three ways to "return" multiple values are: (1) pass pointers/references as parameters, (2) return a struct, (3) return a `std::tuple<>`. Bindings make (2) and (3) painless:

```
std::tuple<std::string, int32_t, double> get_student_data();
auto [name, id, gpa] = get_student_data();
```

They are also handy with map containers, whose insertion methods return an iterator plus a `bool` (Chapters 17 and 18).

`std::optional<>` (C++17, `<optional>`) — represents a value that may or may not exist. Before C++17 you had to return a dummy value like `-1` (ambiguous if `-1` is valid), a pointer with `nullptr`, or a `bool` plus an output reference.

```
std::optional<int32_t> find_first_multiple(const std::vector<int32_t>& vec, int32_t n);
// returns the value if it exists, or std::nullopt if it does not
```

Operation	Behavior
<code>std::optional<T>()</code>	Constructs an optional with no value
<code>std::optional<T>(other)</code>	Copy; no value if <code>other</code> has none
<code>std::optional<T>(value)</code>	Constructs holding value
<code>std::make_optional<T>(value)</code>	Constructs holding value
<code>std::make_optional<T>(args...)</code>	Constructs <code>T</code> in place from constructor arguments
<code>.has_value()</code>	Whether a value is present
<code>.value()</code>	The value, or throws <code>std::bad_optional_access</code>
<code>.value_or(default_value)</code>	The value if present, otherwise <code>default_value</code>

```
std::optional<int32_t> opt1;           // no value
std::optional<int32_t> opt2{42};       // 42
std::optional<int32_t> opt4 = std::nullopt; // no value
auto opt6 = std::make_optional<std::vector<int32_t>>(5, 9); // {9, 9, 9, 9, 9}

std::cout << opt1.has_value() << '\n'; // 0
std::cout << opt1.value_or(281) << '\n'; // 281
if (opt2.has_value()) { std::cout << *opt2 << '\n'; } // check BEFORE using * or ->
```

- `operator*` and `operator->` also access the value but **do not check** first, so using them on an empty optional is **undefined behavior**. Always verify with `.has_value()`. (Dereferencing is more efficient than `.value()` once you have already checked.)

`std::variant<>` (C++17, `<variant>`) — stores one of several possible types in a single variable. Useful when the data's type is not known until it is used (e.g. parsing a file or command line arguments).

```
std::variant<int32_t, std::string> var;
var = 281;                                     // holds the integer
std::cout << std::get<int32_t>(var) << '\n';   // prints 281
var = "EECS";                                 // holds the string
std::cout << std::get<std::string>(var) << '\n'; // prints EECS
std::cout << std::get<int32_t>(var) << '\n';   // throws std::bad_variant_access!

// std::get_if takes an ADDRESS and returns a pointer, or nullptr on a type mismatch
auto* str_value = std::get_if<std::string>(&var); // pointer to "EECS"
auto* int_value = std::get_if<int32_t>(&var);     // nullptr
if (auto* value_ptr = std::get_if<std::string>(&var)) { /* runs if var holds a string */ }
```

- A variant **may not allocate additional dynamic memory** beyond the data it stores, which is a performance benefit (dynamic allocation is more expensive than automatic storage). The stored objects may allocate on their own (e.g. a variant holding a `std::vector<>`); the variant itself may not.
- `std::monostate` can be listed as one of the types to represent a valid **empty** state.

`std::visit()` (C++17) — applies a **visitor** function chosen by the type currently held in a variant. The visitor must be callable with **every** possible type in the variant.

```
struct Visitor {
    void operator()(bool value) { std::cout << "Variant is a bool!\n"; }
    void operator()(const std::string& value) { std::cout << "Variant is a string!\n"; }
};
std::variant<bool, std::string> var1 = std::string{"EECS 281"};
std::visit(Visitor{}, var1); // prints "Variant is a string!"
```

- This supports the **visitor pattern**: separating an algorithm from the objects it operates on, so you can add new behavior (e.g. `calculate_area()` over `Circle`, `Rectangle`, `Triangle`) without modifying the objects themselves. The alternative is plain inheritance with a virtual method in a base class.
- The **overload pattern** is the common idiom: instead of writing a `Visitor` struct, build a visitor out of lambdas using this helper:

```
template<class... Ts> struct overloaded : Ts... { using Ts::operator()...; };
template<class... Ts> overloaded(Ts...) -> overloaded<Ts...>;

double calculate_area(const std::variant<Circle, Rectangle, Triangle>& shapes_variant) {
    return std::visit(overloaded{
        [](const Circle& c) { return std::numbers::pi * c.radius * c.radius; },
        [](const Rectangle& r) { return r.length * r.width; },
        [](const Triangle& t) { return 0.5 * t.base * t.height; }}, shapes_variant);
}
```

`std::any` (C++17, <any>) — stores any copy-constructible type.

```
std::any a = 281;    std::any b = nullptr;
std::any c = "potato"; std::any d = std::vector<int32_t>{1, 2, 3};
```

- **Prefer** `std::variant<>` when you want multiple types in one variable. You still need to know the underlying type to read a `std::any`, and the absence of type restrictions invites runtime bugs ("potato" above is a `const char*`, not a `std::string`, so `std::any_cast<std::string>` would throw).
- `std::any` also lacks the variant's dynamic memory restriction, so it can be less efficient for larger objects.
- It exists as a **safer replacement for** `void*`, which was the pre-C++17 way to hold any type: `std::any` requires you to know the type before accessing the data, while `void*` offers no such guarantee.